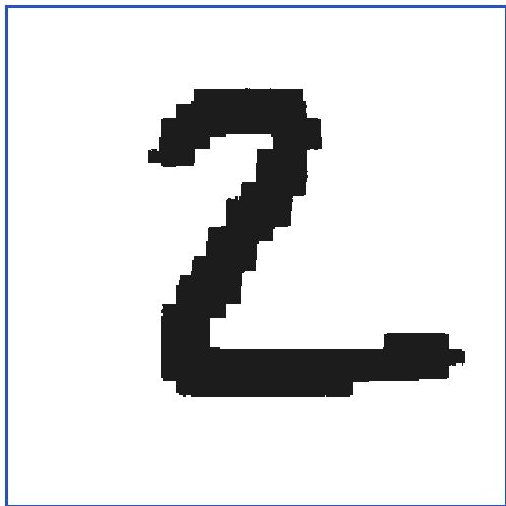
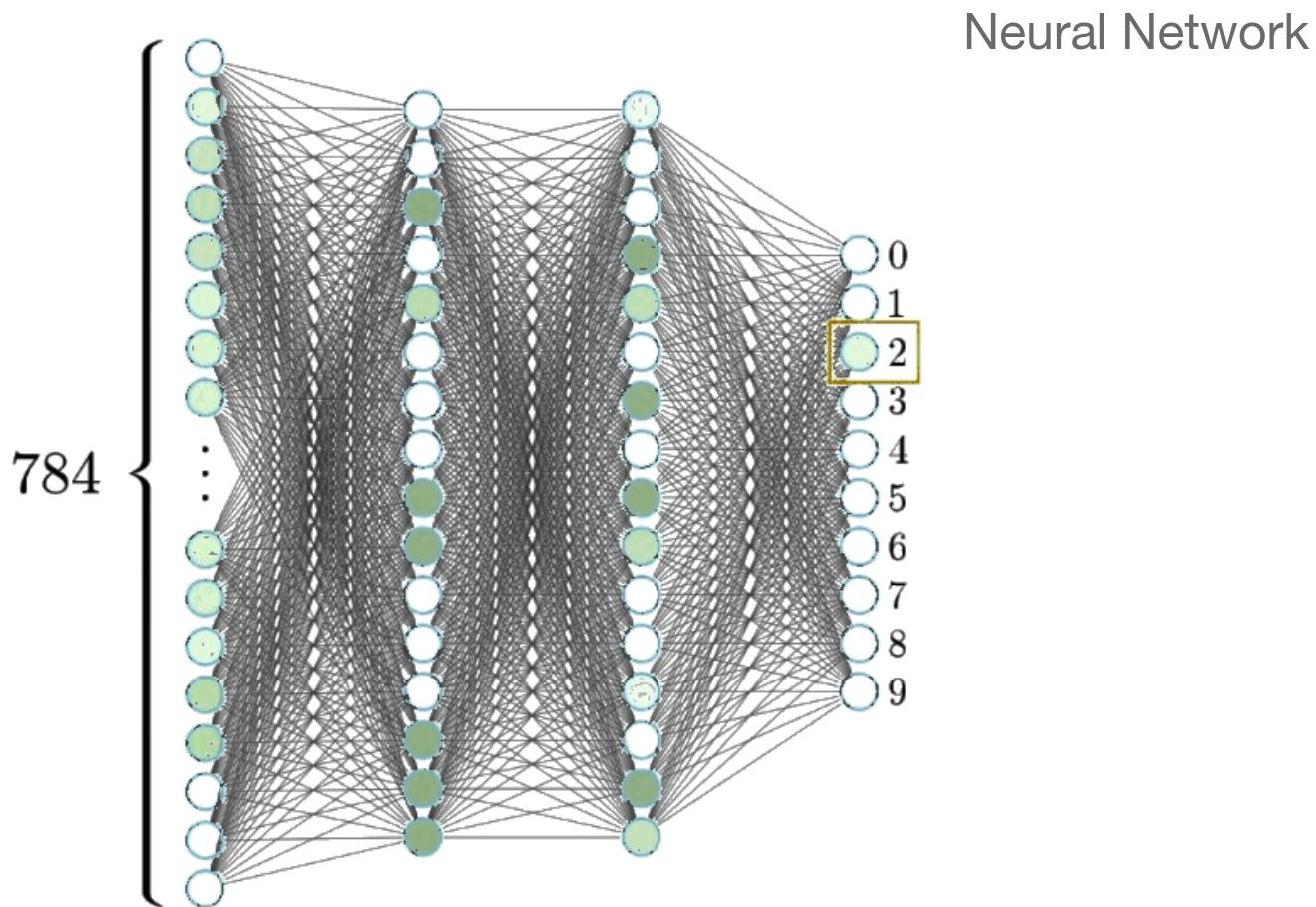




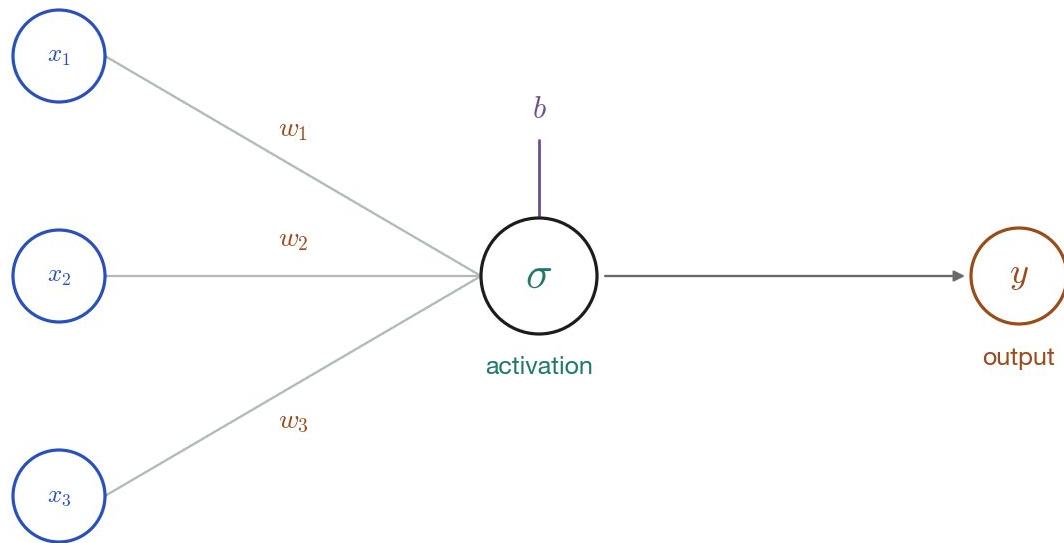
?

- ☐ 0
- ☐ 1
- ☒ 2
- ☐ 3
- ☐ 4
- ☐ 5
- ☐ 6
- ☐ 7
- ☐ 8
- ☐ 9



What is a neural network ?

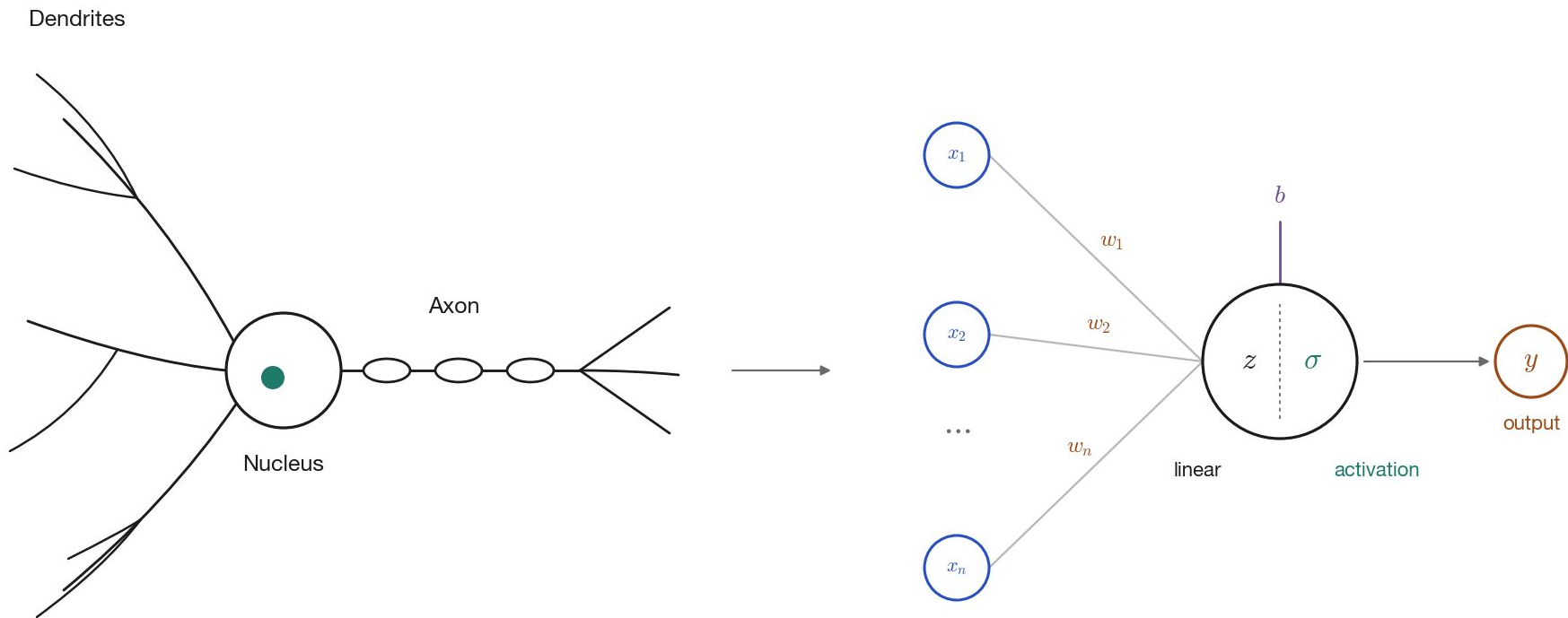
## A neuron

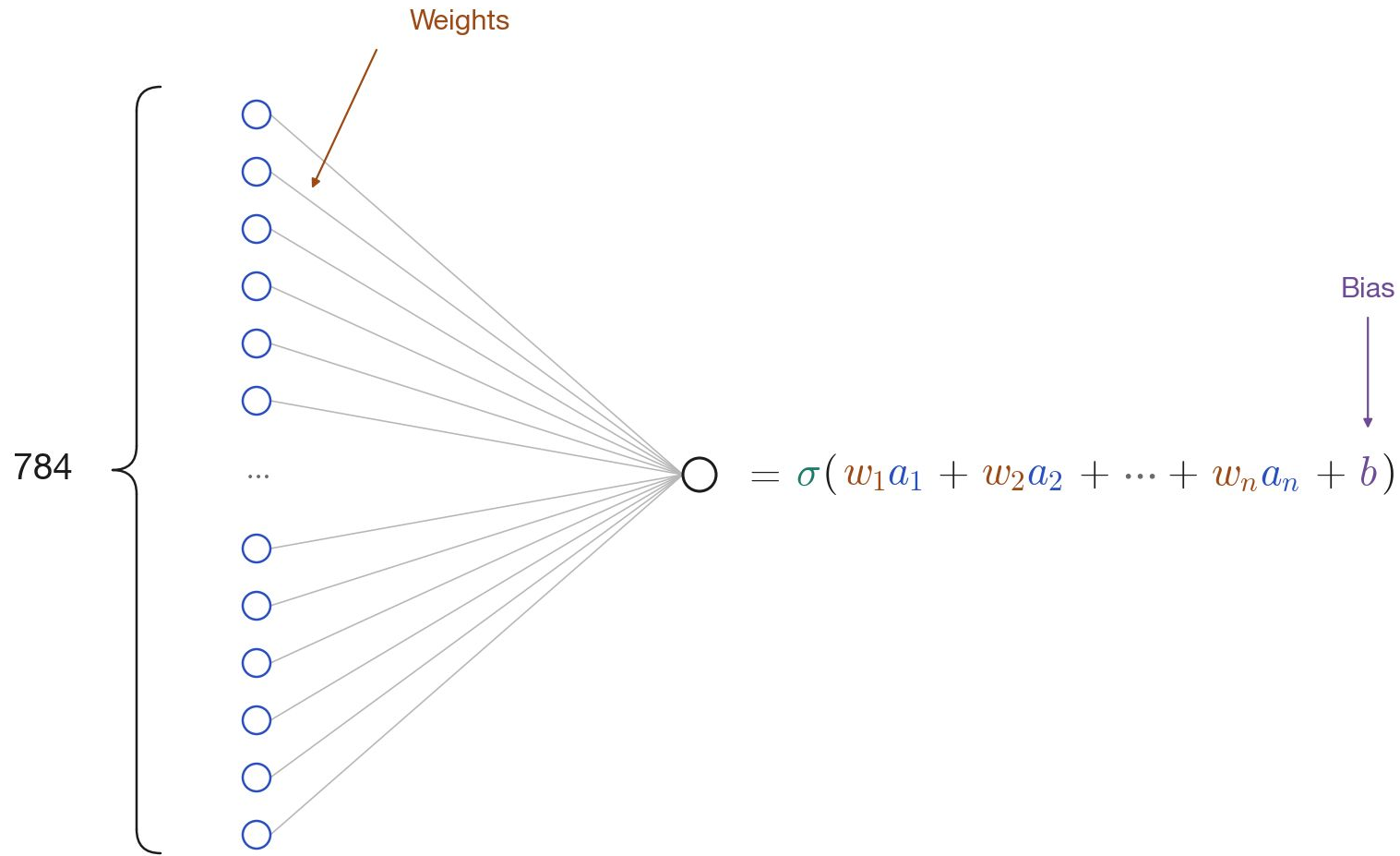


$$y = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

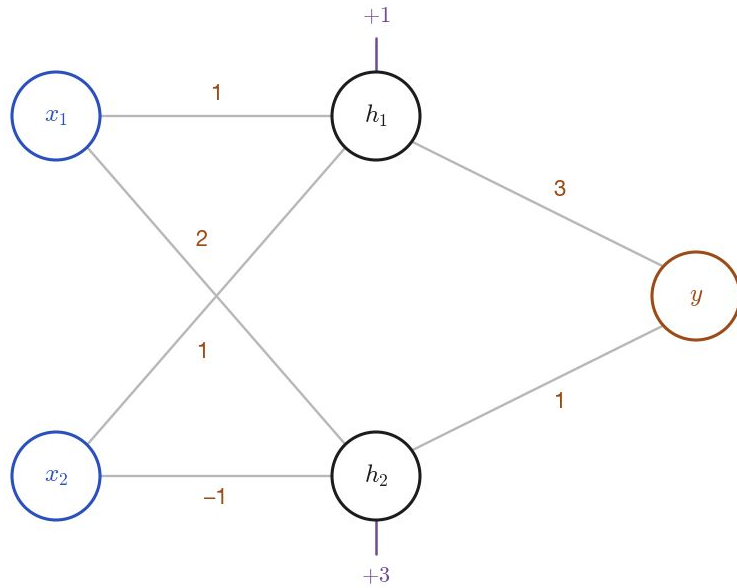
It depends on the **inputs**, the **weights**, and the **bias**.

# Neuron





A small network, with no activation

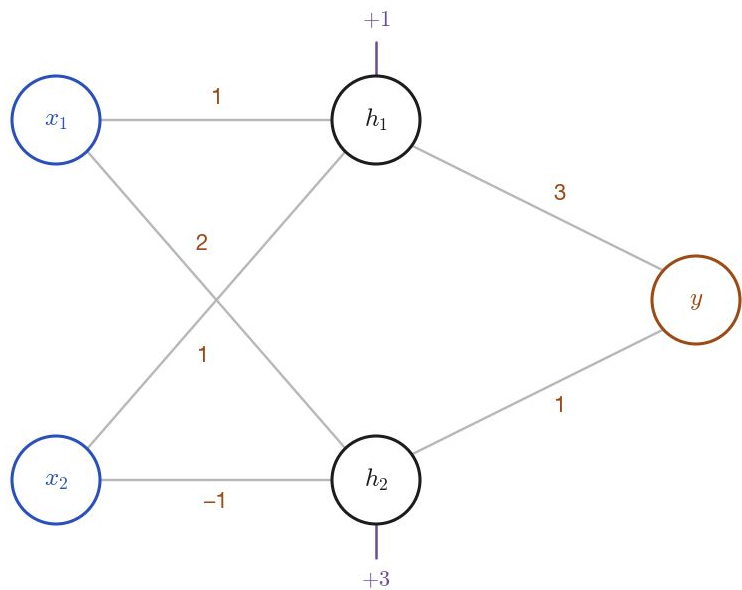


$$h_1 = x_1 + x_2 + 1$$

$$h_2 = 2x_1 - x_2 + 3$$

$$y = 3h_1 + h_2$$

Replace the hidden neurons



$$y = 3h_1 + h_2$$



$$y = 3(\underbrace{x_1 + x_2 + 1}_{h_1}) + (\underbrace{2x_1 - x_2 + 3}_{h_2})$$



Add the matching terms

$$y = 3(x_1 + x_2 + 1) + (2x_1 - x_2 + 3)$$

$$y = 3x_1 + 3x_2 + 3 + 2x_1 - x_2 + 3$$

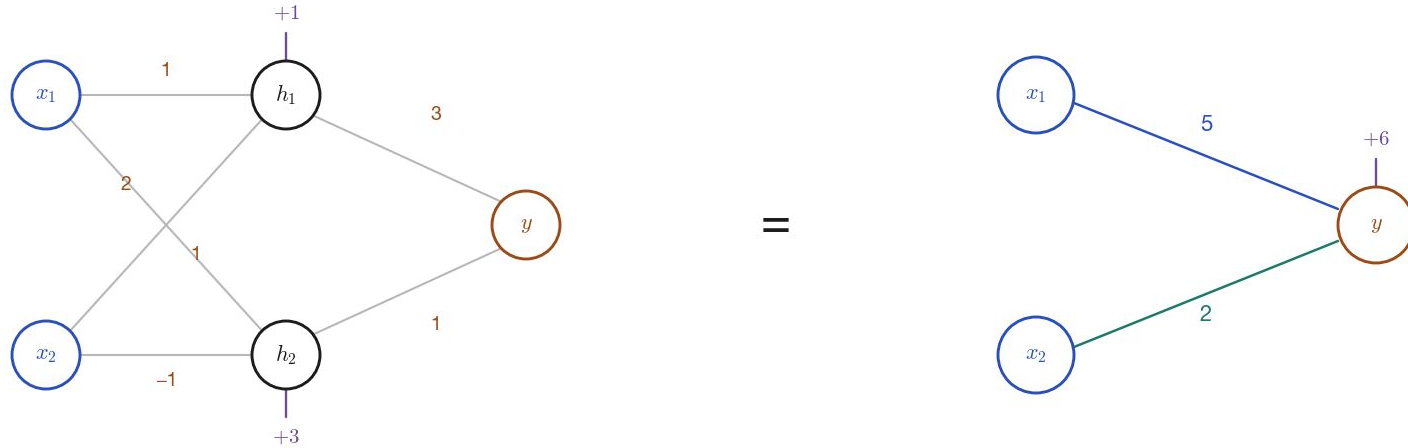
$$3x_1 + 2x_1 = 5x_1$$

$$3x_2 - x_2 = 2x_2$$

$$3 + 3 = 6$$

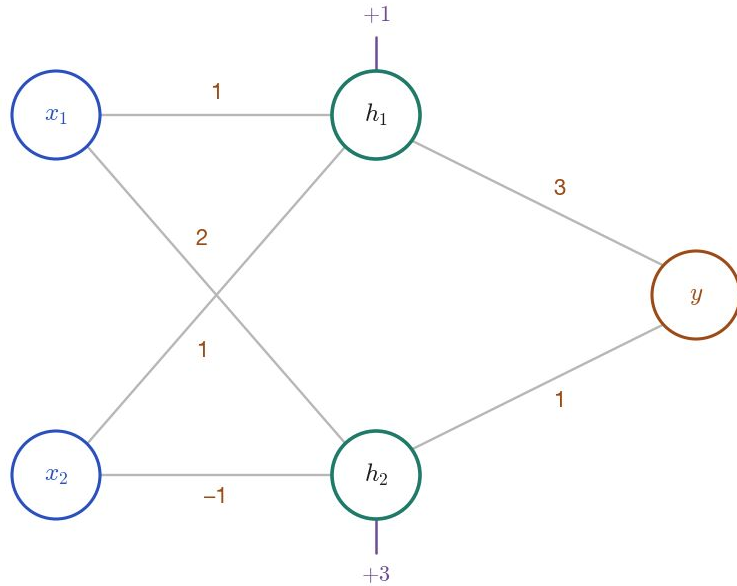
The hidden layer disappears

$$y = 5x_1 + 2x_2 + 6$$



One straight function of the inputs.

Put a nonlinear function on each hidden neuron



$$h_1 = \sigma(x_1 + x_2 + 1)$$

$$h_2 = \sigma(2x_1 - x_2 + 3)$$

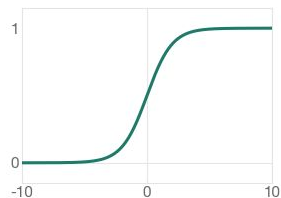
$$y = 3h_1 + h_2$$

The layers no longer collapse into one straight function.

# Activation functions

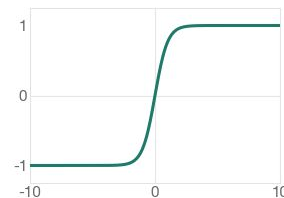
## Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



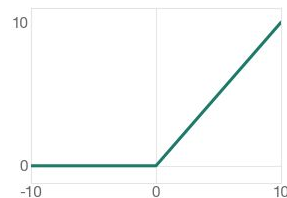
## tanh

$$\tanh(x)$$



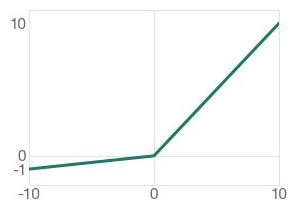
## ReLU

$$\max(0, x)$$



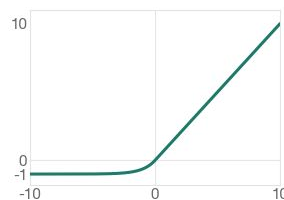
## Leaky ReLU

$$\max(0.1x, x)$$



## ELU

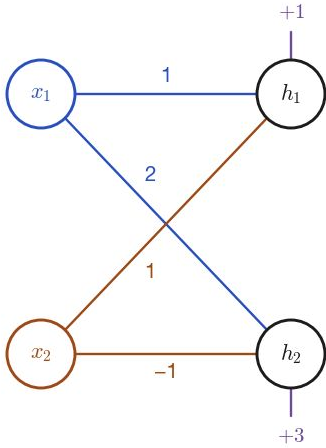
$$\begin{aligned} x & \text{ if } x \geq 0 \\ \alpha(e^x - 1) & \text{ if } x < 0 \end{aligned}$$



## Maxout

$$\begin{aligned} \max(w_1^T x + b_1, \\ w_2^T x + b_2) \end{aligned}$$

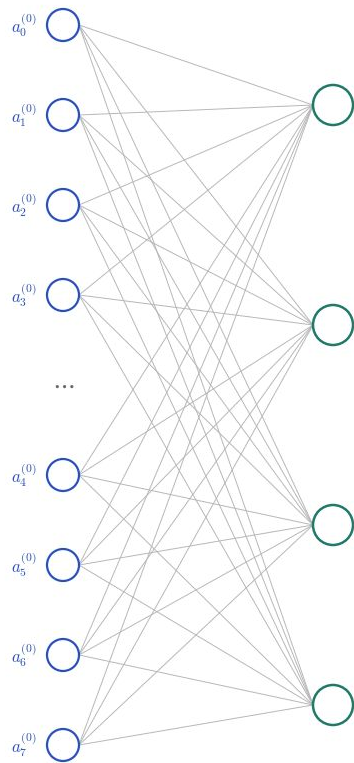
One layer, written as a matrix



$$\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \sigma \left( \begin{bmatrix} 1 & 1 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 \\ 3 \end{bmatrix} \right)$$

neurons                       $W$                        $x$                        $b$   
weights                      inputs                      bias

One row of  $W$  for each neuron. The inputs, and the bias, are columns.



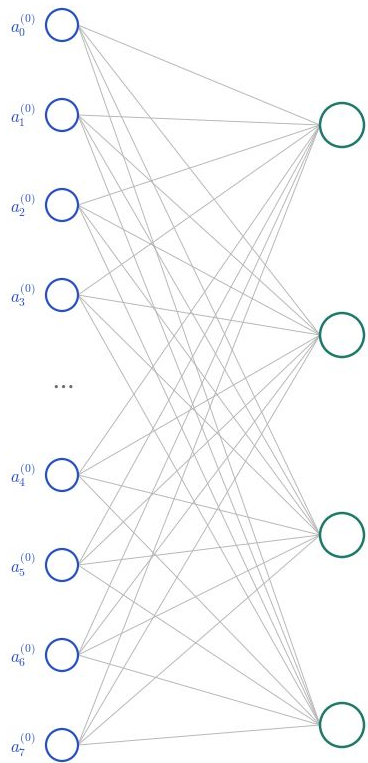
Sigmoid

↓

$$a_0^{(1)} = \sigma(w_{0,0} a_0^{(0)} + w_{0,1} a_1^{(0)} + \dots + w_{0,n} a_n^{(0)} + b_0)$$

Bias  
↙

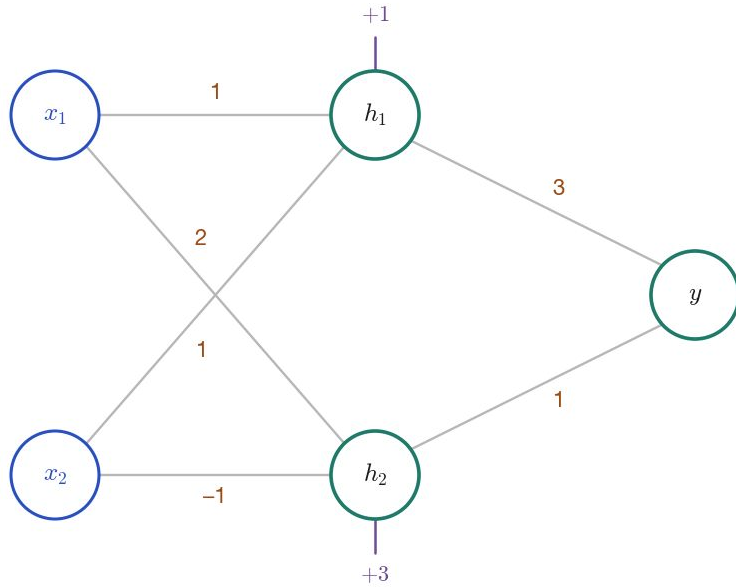
$$\sigma \left( \begin{bmatrix} w_{0,0} & w_{0,1} & \dots & w_{0,n} \\ w_{1,0} & w_{1,1} & \dots & w_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,0} & w_{m,1} & \dots & w_{m,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_m \end{bmatrix} \right)$$



$$\mathbf{a}^{(1)} = \sigma(\mathbf{W}\mathbf{a}^{(0)} + \mathbf{b})$$

```
def feedforward(a):  
    for b, w in zip(biases, weights):  
        a = sigmoid(np.dot(w, a) + b)  
    return a
```

Each layer does the same product



$$h = \sigma(W_1 x + b_1)$$

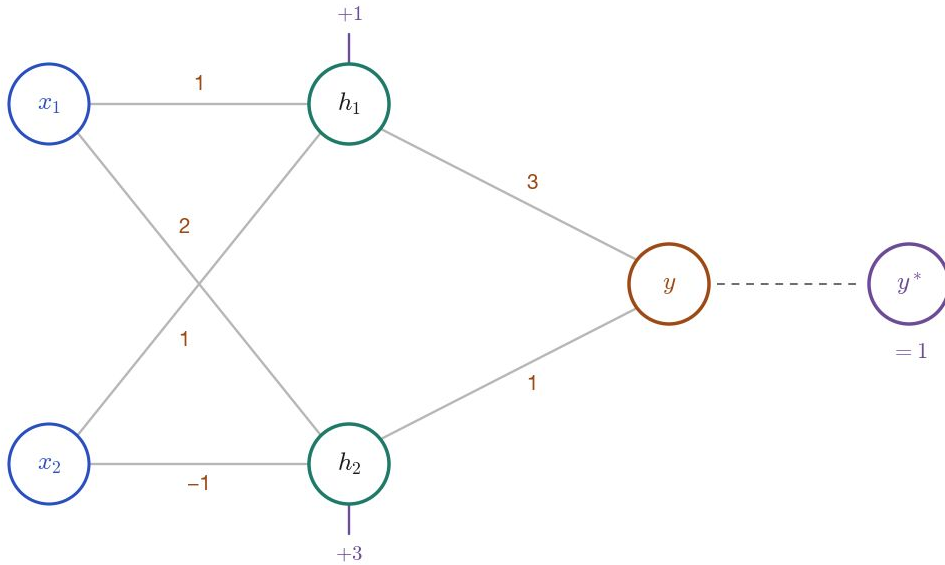
same column

$$y = \sigma(W_2 h + b_2)$$

The column  $h$  leaves the first layer and enters the second.



## The loss



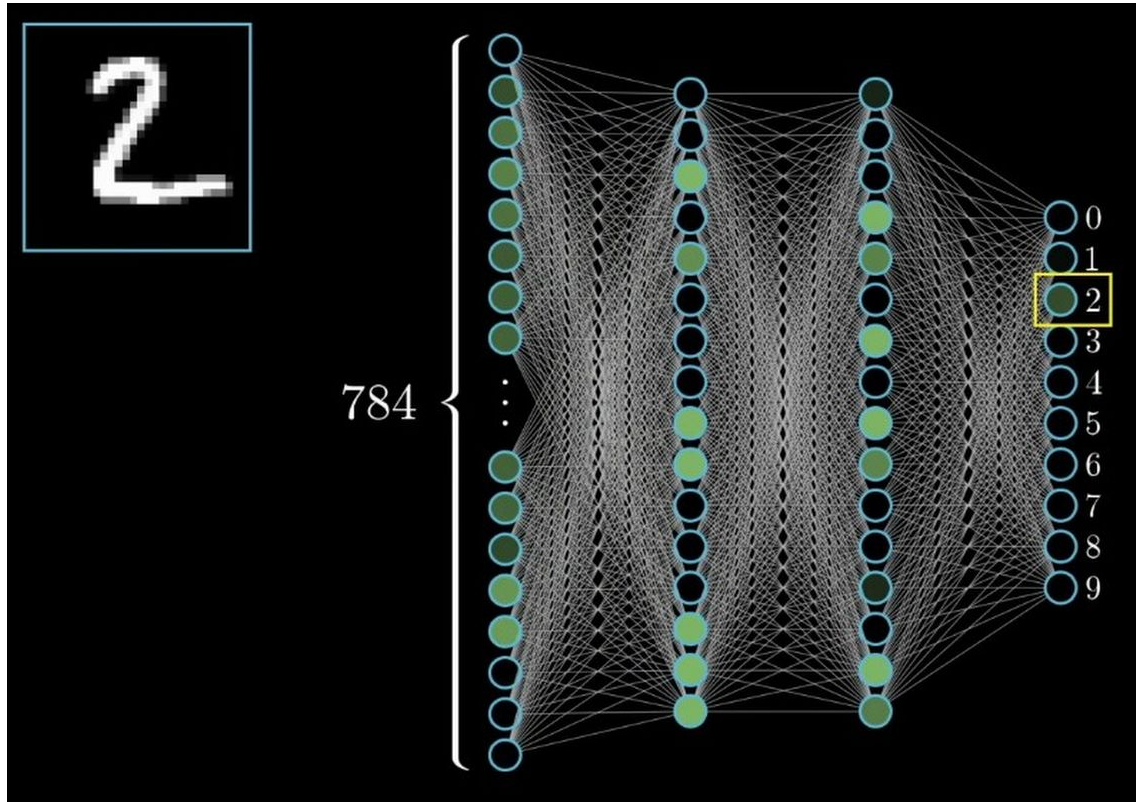
$y$  what this network says

$y^*=1$  the right answer

$$L = \frac{1}{2}(\mathbf{y} - \mathbf{y}^*)^2$$

Zero when the network says 1.

## The loss



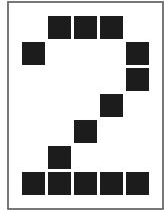
what the network says      the right answer

|                                    |                                      |
|------------------------------------|--------------------------------------|
| <input checked="" type="radio"/> 0 | <input type="radio"/> 0 0            |
| <input checked="" type="radio"/> 1 | <input type="radio"/> 1 0            |
| <input checked="" type="radio"/> 2 | <input checked="" type="radio"/> 2 1 |
| <input checked="" type="radio"/> 3 | <input type="radio"/> 3 0            |
| <input checked="" type="radio"/> 4 | <input type="radio"/> 4 0            |
| <input checked="" type="radio"/> 5 | <input type="radio"/> 5 0            |
| <input checked="" type="radio"/> 6 | <input type="radio"/> 6 0            |
| <input checked="" type="radio"/> 7 | <input type="radio"/> 7 0            |
| <input checked="" type="radio"/> 8 | <input type="radio"/> 8 0            |
| <input checked="" type="radio"/> 9 | <input type="radio"/> 9 0            |

$$L = \frac{1}{2} \sum_{k=0}^9 (y_k - y_k^*)^2$$

For this picture of a 2, the right answer is 1 on the 2 and 0 on the rest.

## Minimize the loss

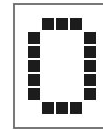
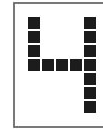
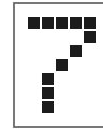


this picture

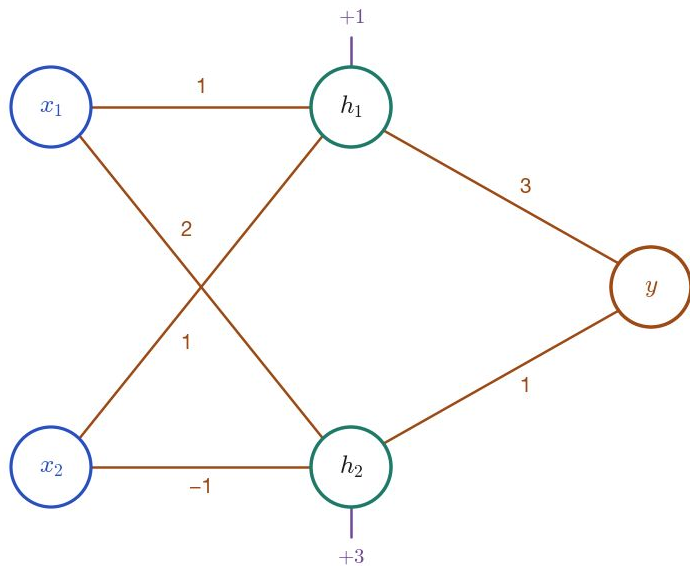


The output moves toward the target.

We hope the same  
for other pictures.



The loss is a function of the picture and the weights



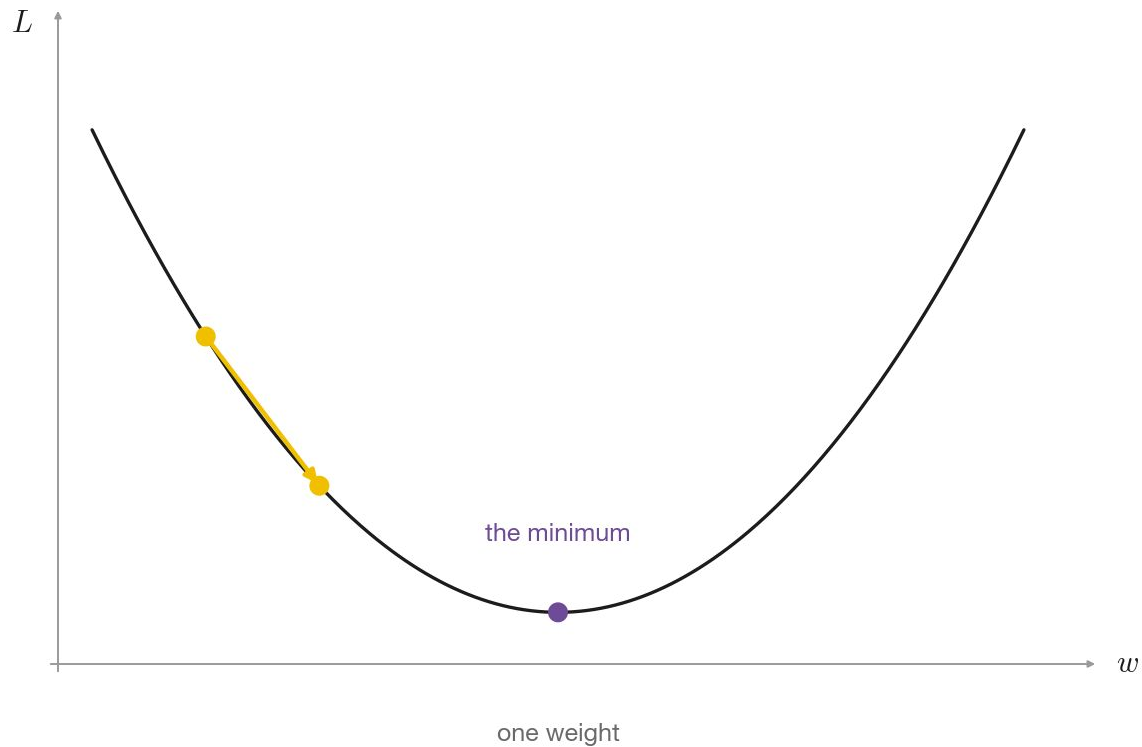
$$L(x, W_1, b_1, W_2, b_2)$$

$x$  the picture. We keep it.

$W, b$  the weights and the biases.

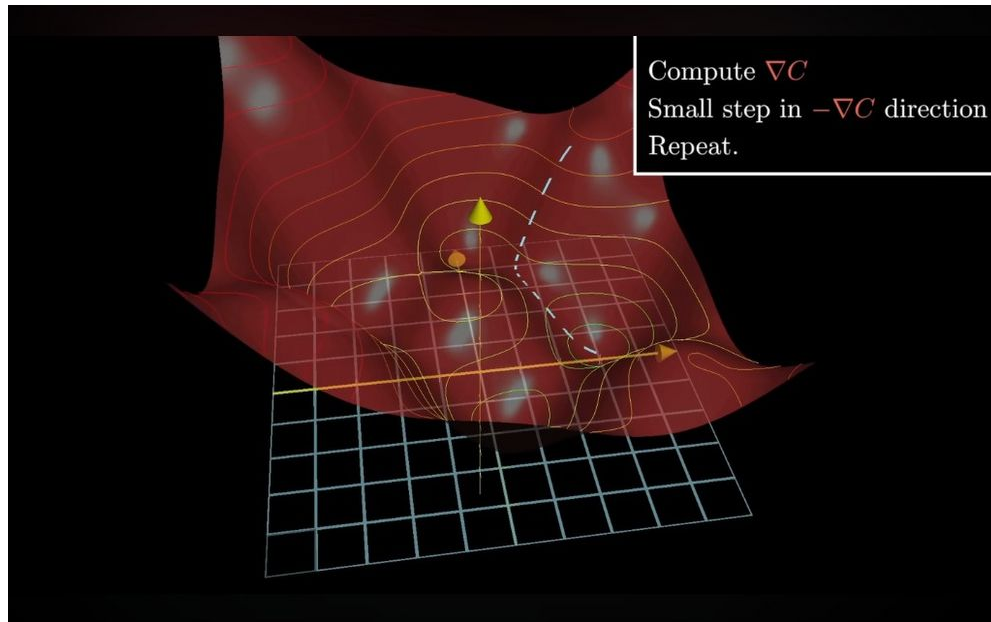
We look for the weights and the biases  
that make  $L$  small.

Go down to the minimum



With one weight, we step toward the bottom.

## Many weights, one loss



$$L : \mathbb{R}^d \rightarrow \mathbb{R}$$

$d$  weights in, one number out. We need the direction where it goes down.

$C$  in this picture is our loss  $L$ .

# The gradient

$$\nabla L = (\partial L / \partial w_1, \partial L / \partial w_2, \dots, \partial L / \partial b_1, \dots)$$

One derivative of the loss for each weight and each bias.

$\partial L / \partial w_1$



$\partial L / \partial w_2$



$\partial L / \partial b_1$

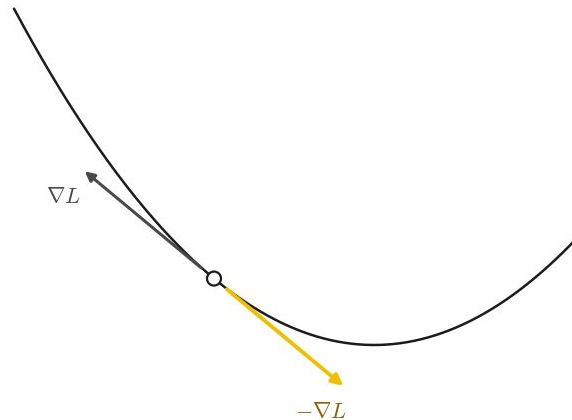


$\partial L / \partial w_3$



$\vdots$

The largest one changes the loss the most.



The gradient points where the loss grows.

We take the opposite.

## The chain rule



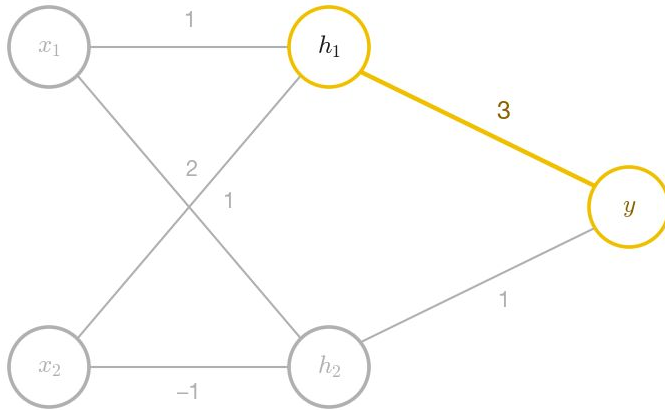
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Multiply the derivatives along the path.



## On the small network

w is this 3



$$z = w h_1 + h_2$$

$$y = \sigma(z)$$

$$L = \frac{1}{2} (y - y^*)^2$$

$$\frac{\partial L}{\partial y} = y - y^*$$

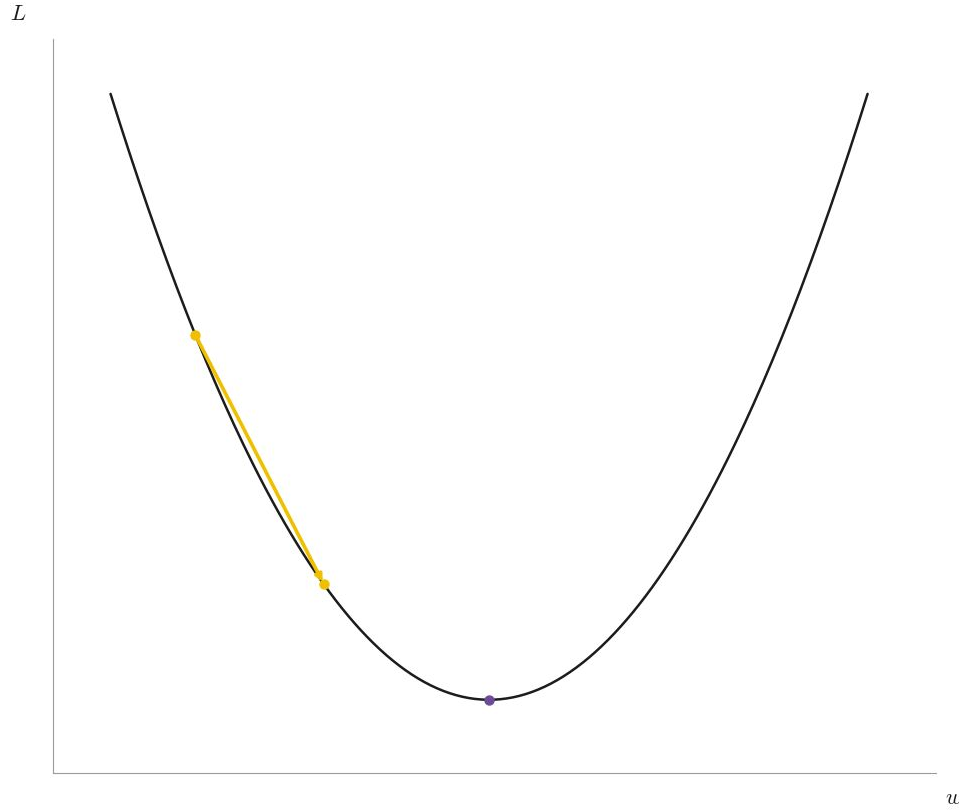
$$\frac{\partial y}{\partial z} = y(1 - y)$$

$$\frac{\partial z}{\partial w} = h_1$$

$$(y - y^*) \cdot y(1 - y) \cdot h_1$$

That product is  $\partial L / \partial w$

## Update the weight

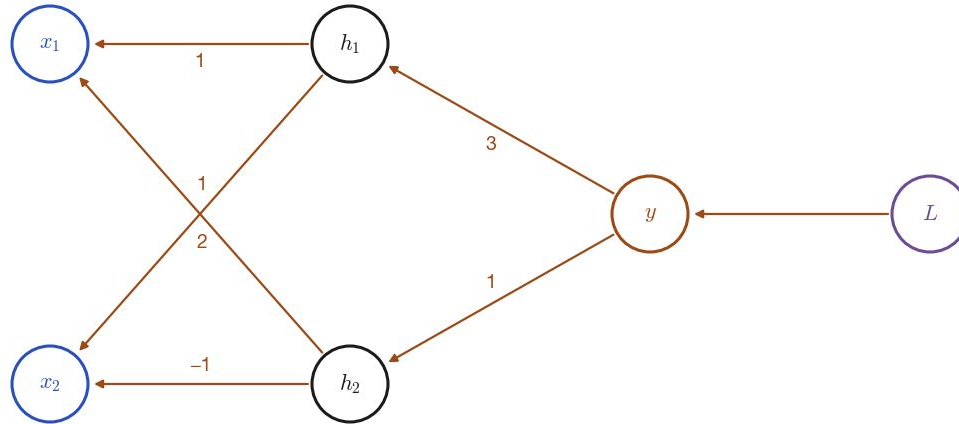


$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

$\eta$  is the size of the step.

We subtract a little of the derivative.

The same product, read backward



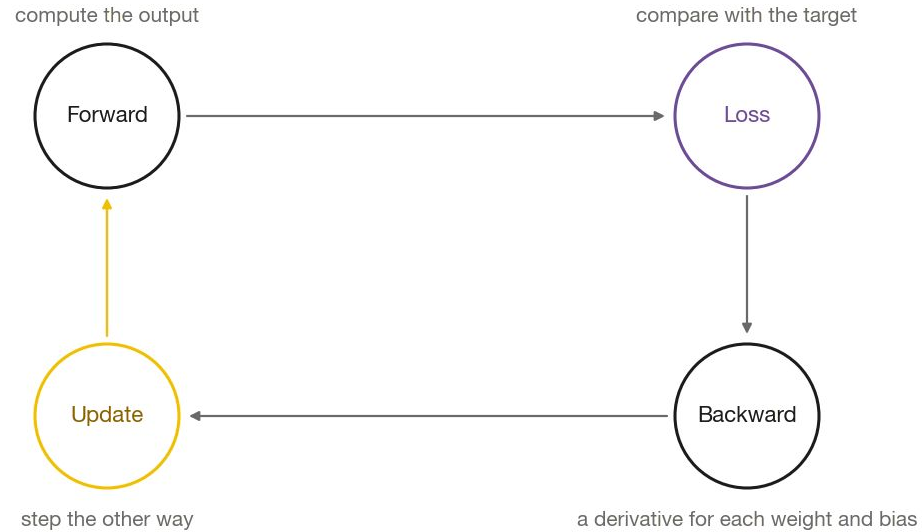
Start at the loss.

Step back along each weight.

Multiply the local slopes.

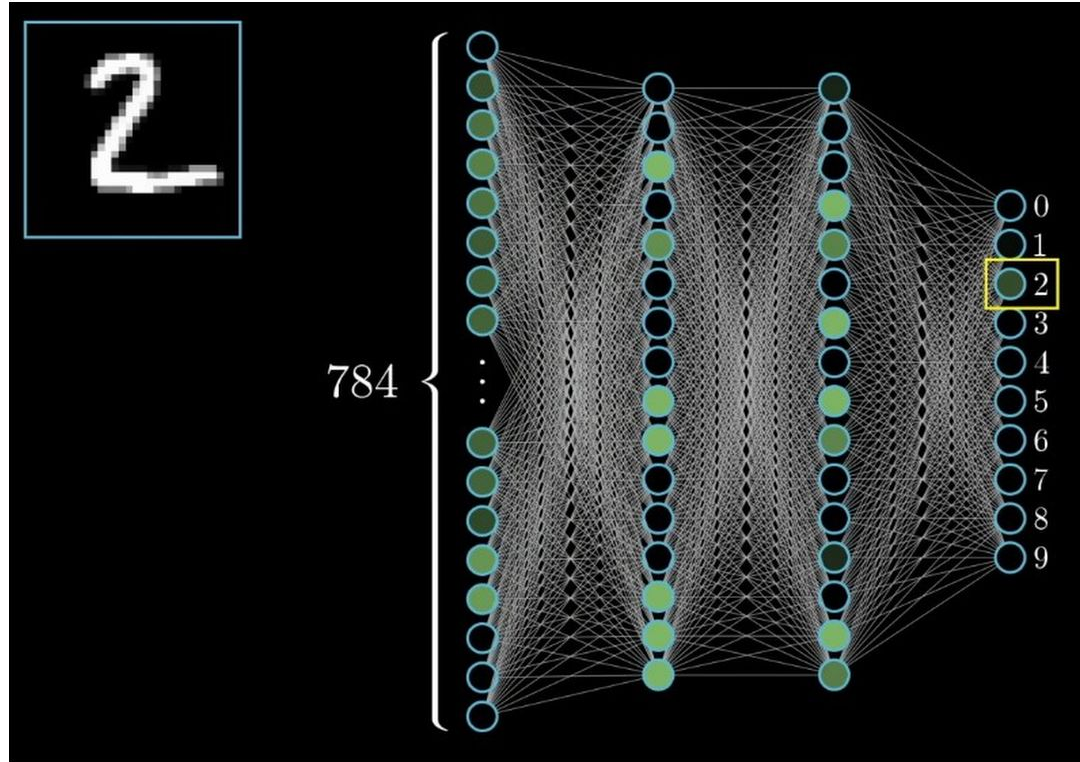
The digit network updates every weight this way.

# The learning loop



Then repeat, with the new weights and biases.

## The digit network



A picture goes in. Ten scores come out. The loop trains every weight and bias.